Project 2 Report: `schedsim`
CMPE 230 Systems Programming – Spring 2026

Yağmur Urtaç, 2024400309
Azelya Altun, 2023400129
2026-04-29

Project Overview

Our project simulates CPU process scheduling by reading a single line of input and generating a clock-by-clock execution schedule according to the specified algorithm. It includes First Come First Serve (FCFS), Shortest Job First (SJF), Shortest Remaining Time First (SRTF), Priority First (PF), and Round Robin (RR) algorithms. We read the input from stdin, then parse the input. We store each process's ID, burst time, arrival time, priority (for PF), and remaining time in a fixed-size buffer. Then, for the simulation, we select the next process at each clock cycle according to the algorithm's rules. We add the selected process's ID (or "X" in case of idle) to output_buf, decrease its remaining time by one, and update current_time. When all processes are complete, we print the output buffer to stdout and terminate the program.

Parsing Logic

Input Processing Strategy

The input line is a structured sequence of tokens where different characters serve as specific boundaries to maintain data hierarchy:

Internal Separation (Hyphens): Hyphens serve as sub-delimiters within a single process. They link a **Process ID** to its specific attributes (Burst, Arrival, Priority), ensuring the parser treats the block as a single entity.

External Separation (Spaces): Spaces act as primary boundaries between independent processes. While hyphens connect data *inside* a process, a space signals the completion of one process and the start of the next.

We parse the input using a pointer stored in `rbx` , which traverses the input buffer character by character. Our code compares the ASCII value of the character in the first memory cell pointed to by this pointer with the ASCII equivalents of the supported algorithms (F, S, P, R). The distinction between SJF and SRTF algorithms, which have the same first character, is resolved using pointer arithmetic. In this case, we check the next byte from the current address with the syntax 1(%rbx); if the second character is 'R', the SRTF algorithm is activated, otherwise the SJF algorithm is activated.

We convert raw ASCII characters into their numeric integer equivalents through a specific sequence of assembly operations:

Character Retrieval: The "movzbq (%rbx), %rdx" command fetches the raw ASCII character from the memory address pointed to by the input buffer pointer.

Digit Verification: Before conversion, the value in the **%rdx** register is compared against the ASCII range for digits (48 for '0' and 57 for '9') to ensure it is a valid numeric character.

ASCII to Integer Conversion: The core conversion is performed using the "subq $48, %rdx" instruction. Since the ASCII table maps the character '0' to the value 48, subtracting this offset transforms the ASCII code (e.g., 53 for '5') into its actual mathematical value (e.g., $53 - 48 = 5$).

Register-Level Parsing Design

%rbx (Input Buffer Pointer): This is the most critical pointer in the code. It is initialized with the command " lea input_buf(%rip), %rbx" . It traverses the raw characters within input_buf step-by-step to perform algorithm detection and data parsing.

%rcx (Process Buffer Pointer): This register holds the base address of the proc_buf area, where parsed data is permanently stored. It is loaded using lea proc_buf(%rip), %rcx and serves as the reference point when writing process attributes such as Burst and Arrival times into memory.

%rsi (System Call Pointer): This register holds the address of input_buf for the sys_read syscall at the beginning of the program. At the end of execution, it holds the address of output_buf for the sys_write syscall.

%r12 (Output Buffer Pointer): This register ensures that every Gantt chart character generated during simulation (e.g., 'A', 'B', or 'X') is written to the correct position within output_buf. It is addressed via lea output_buf(%rip), %r12 and is incremented as each character is written.

%rdx (Dynamic Process Access Pointer): Within the simulation loops, this register points to the specific memory cell of a process. It calculates the direct address of a 16-byte process block by combining the base address (%rcx) with the calculated offset (%r11).

%rax (Accumulator Register): This register serves as the primary accumulator for numeric conversion. It stores the finalized integer value after the digit accumulation process is complete.

%rdx (Data/Character Register): During the parsing loop, %rdx is used to hold the raw ASCII character fetched from the input buffer using movzbq (%rbx), %rdx. It is also the register where the ASCII-to-integer conversion takes place by subtracting 48 (subq $48, %rdx) and is subsequently used to validate if a character is a valid digit (0-9).

%rdi (Process Counter): This register acts as a temporary counter for the number of processes successfully parsed from the input. At the end of the parsing phase, its value is moved to memory (process_count) to define the boundaries of the simulation loop.

%r11 (Memory Offset Calculation): This register is used to store the calculated offset for the process buffer. By shifting the process index in %rdi four bits to the left (shlq $4, %r11), the program computes the exact memory location within proc_buf for each 16-byte process structure.

%r14 (Parameter Constraint): During the parsing phase, this register holds a value (1, 2, or 3) that determines how many numeric attributes must be read for each process based on the selected algorithm.

After the parsing process is complete and the simulation begins, this register holds the index of the process currently occupying the CPU. A value of -1 indicates that the CPU is idle or in a state of selection.

%r15 (Simulation Progress Tracker): During the parsing phase, it holds the algorithm id that is created by us. For example FCFS' id =1 .During the execution of the scheduling algorithms, %r15 stores the number of processes that have finished their execution.

%r8, %r9, %r10 (Comparison Registers): These registers are used within the simulation loops (e.g., in SJF or SRTF) to temporarily hold and compare values like minimum burst time or minimum arrival time to make scheduling decisions.

%r13 (Output Index): This register acts as a global index for output_buf. It tracks the current writing position for the Gantt chart and is finally used as the length parameter (%rdx) for the sys_write system call.

%r12 (Round Robin Quantum Counter): Specifically in the Round Robin implementation, this register serves as a timer that tracks how many time units the current process has spent in the CPU, ensuring it does not exceed the predefined time quantum.

How digits are accumulated into integers?

Initialization: Before each new number is read, the %rax register, where the accumulation process will take place, is reset with the command "xorq %rax, %rax".

Base-10 Scaling: To shift the current numeric value one digit to the left, the value in %rax is multiplied by 10 (imulq $10, %rax).

Digit Addition: The new digit, converted from ASCII to numeral (e.g., a value between 0-9), is added to the %rax register after the multiplication (addq %rdx, %rax).

Loop Termination: This process continues until the pointer (%rbx) encounters a delimiter (space or hyphen) or a newline character. When the loop is finished, the %rax register holds the completed number, and this value is saved to the corresponding memory cell.

Hyphens ('-') as Parameter Delimiters:
When a hyphen character is encountered during a numerical value read loop, this indicates that the parameter (e.g., Burst Time) has ended. The program stops the accumulation process and saves the final value in %rax to memory.Then, the hyphen character is skipped using the "incq %rbx" command, and the pointer is positioned at the beginning of the next numerical value.
Spaces (' ') as Process Delimiters:
Space characters indicate that all data of a process has ended and a new process can begin. The program iterates over the spaces with the %rbx pointer using special routines such as "_skip_tail_spaces" . This process continues until the pointer finds a non-space character (a new Process ID) or a newline character (\n).

Ensuring Data Correctness and Avoiding Corruption

The parsing logic is designed with several low-level safeguards to ensure that process data is accurately translated from the input string to memory without corruption:

- **Zero-Extension for Data Integrity**: When fetching a single byte (character) from the input buffer, the program uses the `movzbq (%rbx), %rdx` command. This performs a "zero-extend" move, which clears the upper 56 bits of the **%rdx** register. This ensures that any "garbage data" left in the register from previous operations does not interfere with the current numeric calculation.
- **Explicit Accumulator Resetting**: Before the start of every numeric parsing loop (for Burst, Arrival, or Priority), the accumulator register (**%rax**) is explicitly cleared using `xorq %rax, %rax`. This prevent values from a previously parsed parameter (e.g., a Process ID) from bleeding into the next parameter's calculation (e.g., Burst Time).
- **Structured Memory Layout (16-byte Alignment)**: The process buffer (`proc_buf`) is organized into fixed 16-byte blocks for each process. By using "`shlq $4, %r11`" ,the program maintains a rigid memory alignment. This prevents "buffer overlapping," ensuring that writing the data for Process B does not accidentally overwrite the attributes of Process A.
- **Boundary Checking and Delimiter Validation**: The parser strictly validates characters against the ASCII range for digits (48–57). If a non-digit character is encountered unexpectedly, the loop terminates immediately (`_read_burst_done`). This prevents the program from attempting to interpret symbols or letters as numeric data, which would lead to logical corruption of the simulation state.

State Representation (Memory Layout)

Process Representation

In this implementation, each process is represented using a **manual struct layout** within a single contiguous memory buffer named `proc_buf`. This design choice enhances data locality and allows for efficient access using pointer arithmetic.

1. Manual Struct Layout (16-Byte Structure)

Each process is allocated exactly **16 bytes** of memory. The internal layout of this "manual struct" is organized as follows to ensure alignment and ease of access:

- **ID (Character):** Stored as a 1-byte ASCII character at **Offset 0** (e.g., 'A', 'B').
- **Priority:** Stored as a 1-byte integer at **Offset 1**. While initialized for all, it is primarily utilized by the Priority (PF) algorithm.
- **Burst Time:** Stored as a 4-byte (long) integer at **Offset 4**, representing the initial CPU time required.
- **Arrival Time:** Stored as a 4-byte (long) integer at **Offset 8**, representing the time the

process enters the system.

- **Remaining Time:** Stored as a 4-byte (long) integer at **Offset 12**. This dynamic field is initialized with the burst time and is decremented by the scheduling algorithms during execution.

2. Memory Organization: Array of Structs

The program effectively implements an **Array of Structs** within the .bss section:

- **Single Unified Buffer:** Rather than using separate parallel arrays (e.g., one array for IDs, another for burst times), all process attributes are interleaved within proc_buf.
- **Pointer Arithmetic for Indexing:** To access the ith process, the program calculates the memory address by multiplying the index by the struct size (16 bytes). In assembly, this is performed using the" shlq $4, %r11" command, which efficiently computes $BaseAddress + (index \times 16)$.

Global State

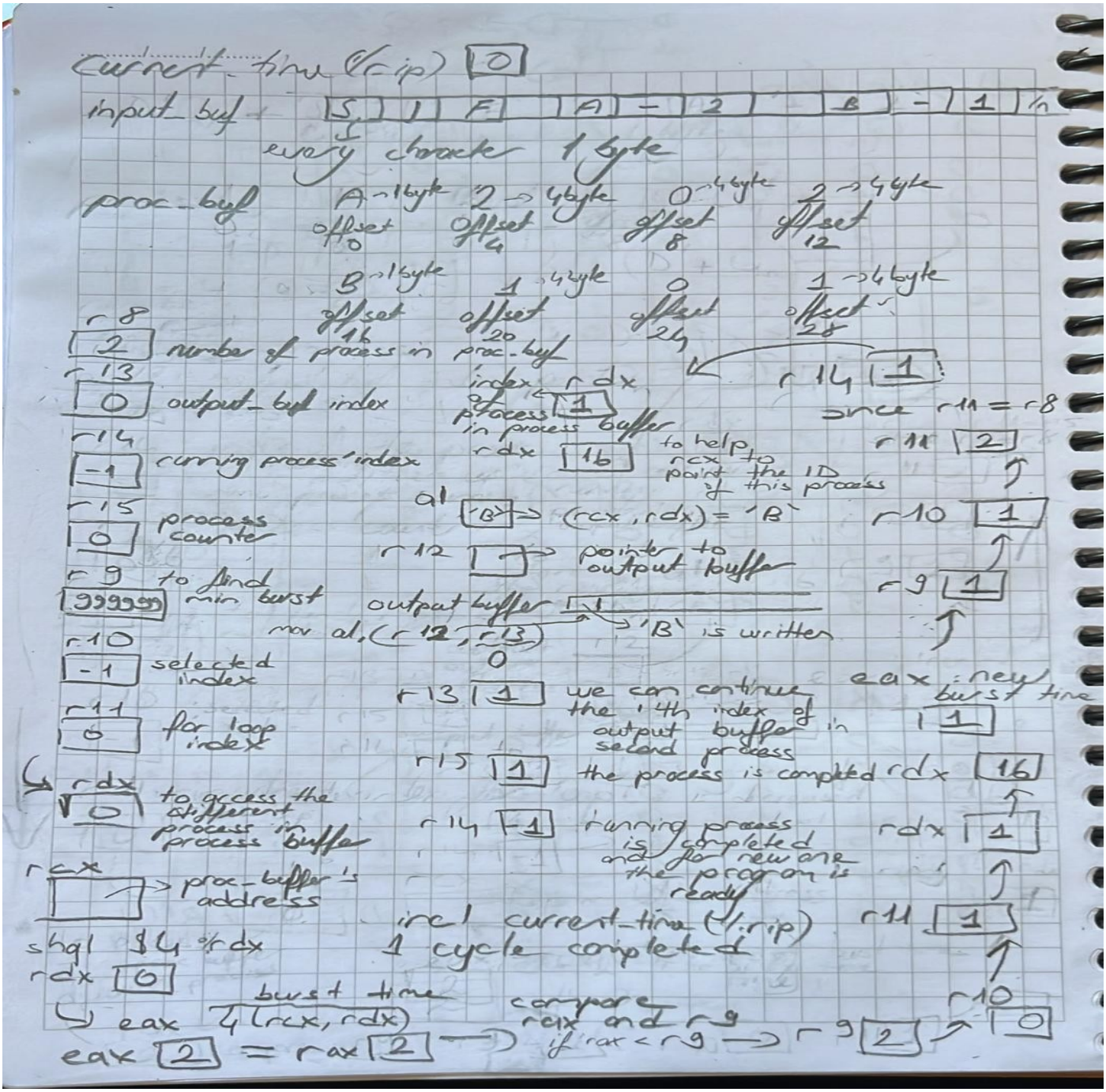
In addition to individual process data, the program tracks global variables to maintain the simulation's timeline and limits:

- **process_count (4 bytes):** Stores the total number of processes identified during the parsing phase.
- **current_time (4 bytes):** Acts as the simulation clock, incremented after every CPU tick or idle period.
- **Ready Queue / Process List:** For general algorithms, proc_buf acts as the primary process list.
- **Round Robin Queue Structure:** For the Round Robin algorithm, a dedicated **rr_queue (64 bytes)** is used alongside rr_head and rr_tail pointers. This creates a **Circular Queue** structure in memory, allowing the program to manage execution order without shifting large blocks of data.
- **I/O Buffers:**

  input_buf: Used for handling raw input data from the system.

  output_buf: Specifically stores the resulting Gantt chart charactersbefore they are printed to the terminal via a system call.

Mandatory Diagram (Hand-Drawn)



Register Usage Strategy

Loop Counters: **%r11** and **%r15** are used as primary counters during simulation loops to track iterations and the number of completed processes.

Pointers: **%rbx** serves as the input buffer pointer, while **%rcx** and **%rsi** manage memory

addresses for the process list and I/O operations.

Current Process Index: **%r14** tracks the index of the process currently occupying the CPU.

Temporary Computations: **%rax** is utilized for primary arithmetic (accumulating digits), and **%rdx** handles intermediate character data and multiplication results.

Preservation and Safety:

- **Preserved Registers:** The design prioritizes the use of **callee-saved** registers (e.g., **%rbx**, **%r12-%r15**) for values that must remain stable across different simulation ticks.
- **How to avoid accidental overwrites:** By assigning fixed roles to specific registers (e.g., **%r13** always tracks the global output length), the program ensures that high-priority state data is never corrupted by temporary calculations.

Conventions:

- The implementation adheres to the **x86-64 Calling Convention**. **%rax**, **%rdi**, **%rsi**, and **%rdx** are treated as **caller-saved** during system calls (like sys_write), while critical simulation states are kept in **callee-saved** registers to minimize stack operations and maintain execution speed.

Scheduling Logic

Clock Cycle Simulation

In each clock cycle, we first determine the eligible process. For this, all processes are scanned, and those satisfying the condition “arrival_time <= current_time” are considered eligible. In algorithms where all processes arrive at time 0, such as SJF and RR, we thought this check unnecessary.

Then, the next process is selected from among the eligible processes according to the algorithm's rule. For FCFS, we looked for the smallest arrival time; for SJF, the smallest burst time; for SRTF, the smallest remaining time; and for PF, we first looked at priority (smaller number means higher priority), and in case of equality, we looked at the remaining time. In RR, we directly selected the process at the beginning of the round-robin queue. If there are no suitable processes, the CPU will go into idle.

After the selection is made, we wrote the ID of the selected process to the output buffer, and "X" if the CPU is in idle.

Finally, we updated the process state by decreasing the remaining time of the selected process by one. If it becomes zero, we considered the process is complete, and we incremented the completed process counter. In RR, we also incremented the quantum counter, and when the quantum was full, we added the process to the end of the queue, incremented current_time by one, and continued this loop until all processes finished.

Algorithm Implementations

FCFS: First, we sorted all processes according to their arrival time. Then, in each cycle, we selected the process with the shortest arrival time among the eligible processes. Then, we executed the selected process until it is complete (until its remaining time=0). As a control flow, we have ensured that if the process is already running, it is executed directly; otherwise, a new process is selected by comparing arrival times in each cycle.

SJF: We implemented a non-preemptive scheduler that selects the process with the shortest burst time when the CPU is idle. First, we reset "current_time" and the process completion counter (%r15). We set the output index (%r13) to the beginning of the output buffer. When the CPU is idle (%r14 = -1), we scan all processes and select the one with the shortest burst time by moving the index from %r10 to %r14. We write the selected process ID to the output buffer and reduce its remaining time. When the process completes (remaining = 0), we increment the process ending counter and set %r14 = -1 to trigger the new selection. We updated current_time globally at each step.

SRTF: We implemented this as the preemptive version of SJF. In each cycle, we scanned all eligible processes and selected the one with the shortest remaining time. Even if one process is already running, we have to prioritize another process if it has less time remaining. So, we performed a fresh selection in every cycle, comparing the running process with others. Then we wrote the selected process's ID to output, and decremented its remaining time. If the ongoing process is the best option, it continues without interruption; otherwise, preemption occurs.

PF: We implemented this algorithm to select the process with the highest priority (the smallest numerical value) in each simulation loop. Since the algorithm is preemptive, the “pf_select_loop” runs at the beginning of each loop, and if a newly arrived operation has a higher priority than the currently running operation, it is immediately prioritized. When two operations have equal priority, we looked at the remaining time as a tie-breaking rule (SRTF logic); we ensured that the operation with the shorter remaining time was selected, and if they were also equal, we preserved the input order. In the register usage, we stored the minimum priority with %r9 (initially 255), the remaining time for tie-breaking with %r10, and the index of the selected operation with %rsi. If a suitable operation could not be found, we switched to the “pf_idle” and wrote "X" to the output buffer. We decreased the remaining time of the selected operation by one with “decl”, and when it reached zero, we incremented the terminated operation counter (%r15). We terminated the simulation when %r15 equaled the total number of operations.

RR: We added all processes to the queue at time 0 in the order of input. Each process must run for a maximum of one quantum. We maintained queue, head, and tail pointers as the control flow. When the quantum is full, we added the process to the end of the queue. If the process finishes within the quantum, we ensured that it fills the remaining slots with "X" and is not added back to the queue. We maintained the quantum counter (r12) as the state update and incremented it in each cycle.

Control Flow Design

Our program consists of four main stages.

Parsing: Parsing starts from the "_start" label. After reading the input, we determined the algorithm by looking at the first character. Then we proceed character by character in the "_parse_loop" label. When we encounter a space or hyphen, we skipped with "incq %rbx". When we encounter a number, we jumped to the "_read_burst_loop". When the parsing is finished, we jumped to the "_simulation_start" label.

Scheduling: Depending on the algorithm type, we went to one of the labels we had separated, such as "_run_fcfs", "_run_sjf". We designed each algorithm to have its own main loop. In each loop, a process is selected first (e.g., "fcfs_select_loop"), then the selected process is run (e.g., "fcfs_run_current"), and if no process is selected, it goes to an idle (e.g., "fcfs_idle"). At the end of the loop, the current_time is updated and the cycle returns to the beginning. We controlled all loops with conditional jumps.

Output: When all processes are finished, the program goes to the “_output_final” label. Here, we added a newline to the end of the output buffer and printed to stdout using “syscall”.

Termination Condition: We kept a counter (%r15) for each algorithm's completed processes. %r15 is incremented each time a process finishes. When %r15 equals the total number of processes, the loop exits and goes to “_output_final”. The program ends with the “.exit” label.

Label and Jump Usage: We used labels to mark different sections of the code. We created decision mechanisms with conditional jumps and used unconditional jumps (jmp) to return through loops and skip spaces/hyphens during parsing. We used a counter-based for-loop pattern. The counter register holds the starting value, the label holds the comparison, then the counter is incremented, and a return is made using jmp.

Representative Code Snippet

```
srtf_select_loop:
    cmpq    %r11, %r8
    je      srtf_select_done

    movq    %r11, %rdx
    shlq    $4, %rdx            # 16 byte offset
    lea     proc_buf(%rip), %rcx

    # kalan süre > 0 mı?
    movl    12(%rcx, %rdx), %eax
    cmpl    $0, %eax
    jle     srtf_select_next

    # geldi mi? (yani Arrival <= CurrentTime mı)
    movl    8(%rcx, %rdx), %ebx
    cmpl    current_time(%rip), %ebx
    jg      srtf_select_next

    # kalan süresi daha mı kısa?
    cmpl    %eax, %r9d
    jle     srtf_select_next

    movl    %eax, %r9d          # yeni minimum kalan süre
    movq    %r11, %r10          # yeni seçilen index

srtf_select_next:
    incq    %r11
    jmp     srtf_select_loop
```

This loop scans all processes up to current_time and finds the process with the smallest remaining time among those whose remaining time is greater than zero. The index of the selected process is stored in %r10, and its remaining time is stored in %r9.

Register Usage:

%r8 = total number of processes (process_count)

%r11 = cycle counter (i)

%rdx = offset calculation (i * 16)

%rcx = proc_buf base address

%eax = temporary remaining time

%ebx = temporary arrival time

%r9d = smallest remaining time found so far

%r10 = index of the process with the smallest remaining time

Control flow: The loop starts with “cmpq %r11, %r8”. The loop exits if all processes have been scanned. For each process, a remaining time check (cmpl $0, %eax) is performed, followed by an arrival time check (cmpl current_time, %ebx). For processes that meet both conditions, a comparison of remaining time is made using “cmpl %eax, %r9d”. If a smaller value is found, %r9d and %r10 are updated. At the end of the loop, the counter is incremented, and the loop returns to the beginning with jmp.

Why it's correct: Because the loop scans every process, and only evaluates the eligible ones. Since jle is used for comparison, no updates are made in cases of equal remaining times. This ensures that among processes with the same remaining time, the one that came first in the input order is selected. This implements the tie-breaking rule correctly.

Testing and Debugging

Testing Strategy

While testing, we used sample inputs that were given in the description file. We created edge cases our own and did manual testing. We also used the given grader file in order to do automated testing.

Debugging Approach

In some places, we used syscall to print selected process ID, current time, and remaining

time values for debugging purposes. We removed those lines before submitting the final code. We examined the register values using gdb at critical points. For example; rbx in _parse_loop, rax during multi-digit number conversion, r12 in RR. Also, we manually traced algorithms cycle-by-cycle and compared with expected output. We caught some bugs in PF tie-breaking, RR quantum reset, and SRTF arrival checks.

Edge Cases

Multiple arrivals at the same time: FCFS A-3-1 B-2-1 C-4-1	Output: XAAABBCCCC
Identical priorities: PF A-3-0-2 B-4-0-2 C-2-0-2	Output: CCAAABBBB
Quantum partially used: RR A-4 3	Output: AAAAXX
CPU idle periods: FCFS A-3-2	Output: XXAAA

Course Concepts

Memory and Data Layout

We used parallel arrays approach to store process data. We stored each process in a 16-byte block (Process ID: 1 byte, Priority: 1 byte (just for PF), 2 byte padding, Burst time: 4 byte, Arrival time: 4 byte, Remaining time: 4 byte). We arranged these blocks sequentially within “proc_buf”. To access a process, first we get the index, then multiply it by 16 and then add it to the base address.

Other buffers are input_buf, output_buf and rr_queue. “input_buf” is 256 byte, we used it for reading a line from stdin. “output_buf” is 1024 byte, we used it to put output characters. “rr_queue” is 64 byte, we used it for round-robin queue for RR algorithm.

Registers and Low-Level Computation

We stored everything in the registers as requested in the description file. “rbx” is input buffer pointer, “rax” is accumulator, “r8” is process_count, “r9” is minimum value, “r10” is selected index, “r12” is RR quantum counter, “r13” is output buffer index, “r14” is algorithm type, “r15” is the number of completed processes. To read a multi-digit number we did rax = rax * 10 + (char – '0') as shown below.

```
# Yeni rakam okuduğunda eskisini 10 ile çarpıp yenisini ekliyor.
subq    $48, %rdx           # karakteri rakama çevir
imulq   $10, %rax
addq    %rdx, %rax
```

Control Flow and Branching

We did all control flow using labels and conditional jumpers (je, jne, jl, jg, jle, jge). We maintain a counter register (r11) to implement for loops. Then we perform comparisons in the label (e.g., cmpq %r11, %r8), if they are equal (je) we exit the loop, otherwise we perform the operations, increment the counter and jump back.

To implement decision-making mechanisms, we established the selection logic of each algorithm using nested conditional jumps. For example, in PF, we first compared priority (with jl for higher priority). If priorities were not equal, we jumped ahead (with jne); if they were equal, we continued and compared remaining time.

Parsing Without Libraries

We parsed the strings character by character at the register level without using any C library functions.

We used the “rbx” pointer to navigate the input buffer. We parsed the characters one by one as follows:

1.) We looked at the first character (F, S, P, R) to determine the required algorithm. If it was S, we looked at the next character to decide whether it was SJF or SRTF.

2.) We skipped the spaces and hyphens (-) with “incq %rbx”. Hyphens indicate the end of a field and the start of the next one within the same process.

3.) Since it was a single character, we wrote the process ID directly to the “proc_buf”.

4.) We read the numbers by calculating %rax = %rax * 10 + (char - '0') as we encountered digits. We stopped when a hyphen, space, or newline appeared.

5.) We kept it at r14 according to the algorithm. SJF has 1 number (burst), FCFS and SRTF have 2 numbers (burst, and arrival), and PF has 3 numbers (burst, arrival, and priority).

Tradeoffs

In the simplicity vs efficiency comparison,, we prioritized simplicity. By scanning all processes in each scheduling cycle (O(n)), we captured the smallest remaining time or earliest arrival. A more efficient data structure (e.g., a priority queue) could have been used, but implementing it in assembly would have been too complex for us. Since the number of processes is small, we thought O(n) is sufficient.

In the readability vs performance comparison, we prioritized readability. We wrote each algorithm separately (_run_fcfs, _run_sjf, _run_srtf, etc.). We could have written a common scheduler and achieved different behaviors with register settings, but this would have made the code much harder to read and debug. We also tried to keep register usage consistent (e.g., r9 always the minimum value, r10 the selected index). In this way, we tried to minimize the

performance loss.

In the static vs dynamic design comparison, we chose static. We defined buffers as fixed-size (proc_buf: 160 bytes, output_buf: 1024 bytes, rr_queue: 64 bytes). We couldn't use malloc for dynamic memory management (because we can't use a C library), and making syscalls with brk would be too complicated. Since the specification provides limits for the process number, we thought the static design is sufficient and safe.

Conclusion

The strongest design decision: Writing each scheduling algorithm separately (_run_fcfs, _run_sjf, etc.). This made our code more readable and easier to debug, and allowed us to test each algorithm independently. Consistent register usage across algorithms (e.g., r9 for minimum value, r10 for selected index) also improved readability.

The biggest challenge: Implementing the PF tie-breaking. The specification requires comparing priority first, then comparing remaining time if priorities are equal, and finally falling back to the input order. While implementing nested conditional jumps correctly, the debugging process was challenging.

Possible improvements: Using a more efficient data structure like a priority queue instead of iterating through all processes in each loop for SRTF. However, this significantly increases code complexity and is hard to implement.

Overall, our project successfully simulates all five scheduling algorithms in assembly language, produces correct outputs for the provided test cases.

AI Usage Statement

We used AI tools (ChatGPT and Gemini) as assistants while writing code and as helpers in the debugging process. We confirm that we understood and have written every piece of code in the project.